

✓ Python Regular Expressions (Regex) Tutorial

This tutorial will guide you through the basics of Regular Expressions (Regex) in Python, as well as provide some advanced examples. It will also connect some concepts of lexical analysis (as in compiler design) to how we use these patterns for text matching in Python.

Note: You can run each code block in Google Colab or any other Python environment.

Table of Contents

1. [Introduction to Regular Expressions](#)
 2. [Basic Regex Syntax](#)
 3. [Python's `re` Module](#)
 4. [Common Regex Functions in Python](#)
 5. [Regex Examples and Exercises](#)
 6. [Advanced Use Cases](#)
 - [Password Validation](#)
 - [Email Validation](#)
 - [URL Validation](#)
 7. [Connecting Regex to Lexical Analysis](#)
 8. [Conclusion](#)
-

1. Introduction to Regular Expressions

Regular Expressions are a powerful way to describe patterns in text. They are used in many fields such as:

- Text processing
- Data cleaning
- Log file analysis
- Validation of input (e.g., email, password, etc.)
- **Lexical analysis** in compilers, where they help describe tokens in a programming language.

A **regex** can be seen as a formal language that specifies patterns to match. For example, the pattern `[A-Za-z]+` matches one or more alphabetic characters.

2. Basic Regex Syntax

Below are some of the most common elements you will see in regular expressions:

Symbol/Pattern	Meaning	Example
<code>.</code>	Matches any character (except newline)	<code>a.c</code> matches <code>abc</code> , <code>adc</code> , etc.
<code>^</code>	Start of string	<code>^hello</code> matches <code>hello</code> at start only
<code>\$</code>	End of string	<code>world\$</code> matches <code>world</code> at end only
<code>*</code>	0 or more occurrences	<code>ab*</code> matches <code>a</code> , <code>ab</code> , <code>abb</code> , etc.
<code>+</code>	1 or more occurrences	<code>ab+</code> matches <code>ab</code> , <code>abb</code> , etc.
<code>?</code>	0 or 1 occurrence	<code>ab?</code> matches <code>a</code> or <code>ab</code>
<code>{n}</code>	Exactly n occurrences	<code>a{3}</code> matches <code>aaa</code>
<code>{n,}</code>	n or more occurrences	<code>a{2,}</code> matches <code>aa</code> , <code>aaa</code> , <code>aaaa</code> , etc.
<code>{n,m}</code>	Between n and m occurrences	<code>a{2,4}</code> matches <code>aa</code> , <code>aaa</code> , <code>aaaa</code>
<code>[]</code>	Character class (set of allowed chars)	<code>[abc]</code> matches <code>a</code> , <code>b</code> , or <code>c</code>
<code>[^]</code>	Negated character class	<code>[^abc]</code> matches any char not <code>a, b, c</code>
<code>\</code>	Escape character or special sequences	<code>\d</code> matches digits, <code>\s</code> whitespace, etc.
<code> </code> (pipe)	OR operation	<code>cat dog</code> matches <code>cat</code> or <code>dog</code>
<code>()</code>	Grouping	<code>(abc)+</code> matches one or more <code>abc</code>

Some **special sequences** you'll frequently use:

- `\d`: Matches any digit (equivalent to `[0-9]`).
- `\D`: Matches any non-digit (equivalent to `[^0-9]`).
- `\s`: Matches any whitespace character (space, tab, newline).
- `\S`: Matches any non-whitespace character.
- `\w`: Matches any alphanumeric character (digits, letters, underscore).
- `\W`: Matches any non-alphanumeric character.

3. Python's `re` Module

In Python, regular expressions are supported by the built-in module `re`.

To start using regex in Python, you typically do:

```
import re
```

4. Common Regex Functions in Python

1. `re.match(pattern, string)`

- Tries to match the pattern at the **beginning** of the string.
- Returns a match object if successful, else `None`.

2. `re.search(pattern, string)`

- Scans through the entire string, looking for **any location** where the pattern matches.
- Returns the first match object if successful, else `None`.

3. `re.findall(pattern, string)`

- Returns a **list** of all non-overlapping matches of pattern in the string.

4. `re.sub(pattern, repl, string)`

- Replaces the leftmost non-overlapping occurrences of pattern in the string with `repl`.
- Returns a new string.

5. `re.split(pattern, string)`

- Splits the string by occurrences of the pattern.
- Returns a list of substrings.

6. `re.compile(pattern)`

- Compiles a regex pattern into a regex object for reuse.
- Useful if you are using the same pattern multiple times.

5. Regex Examples and Exercises

Run the following examples in a Google Colab cell (or any local Python environment):

Example 1: Basic `re.search()`

```
import re

text = "Hello world!"
pattern = r"world" # raw string notation recommended for regex

match = re.search(pattern, text)
if match:
    print("Pattern found at index:", match.start())
```

```
else:  
    print("Pattern not found.")
```

Example 2: `re.match()` vs. `re.search()`

```
import re  
  
text = "Hello world!"  
pattern = r"Hello"  
  
match_result = re.match(pattern, text)  
search_result = re.search(pattern, text)  
  
print("match() result:", match_result) # Will not be None if '  
print("search() result:", search_result) # Will find 'Hello' any
```

Example 3: Finding All Occurrences using `re.findall()`

```
import re  
  
text = "Cats are smarter than dogs. Dogs are amazing!"  
pattern = r"[Dd]ogs?" # Matches 'Dog' or 'dog' and optionally t  
  
matches = re.findall(pattern, text)  
print("All matches:", matches)
```

Example 4: Substituting text using `re.sub()`

```
import re  
  
text = "My phone number is 123-456-7890."  
pattern = r"\d" # Digit
```

```
# Replace all digits with 'X'
result = re.sub(pattern, "X", text)
print("Substituted text:", result)
```

Example 5: Splitting text using `re.split()`

```
import re

text = "one,two three:four|five"
# Split on either comma, space, colon, or pipe
pattern = r"[,\s:|]+"

split_result = re.split(pattern, text)
print("Split result:", split_result)
```

6. Advanced Use Cases

Password Validation

A typical password pattern might require:

1. At least 8 characters
2. Contains both letters and numbers
3. (Optionally) Contains special characters

For example:

```
import re

def validate_password(password):
    # Example pattern:
    # ^                - start of string
    # (?=.*[A-Za-z])   - at least one letter
    # (?=.*\d)         - at least one digit
    # [A-Za-z\d@$!%*?&] - allowed characters
    # {8,}             - minimum length of 8
```

```

# $ - end of string
pattern = r"^(?=.*[A-Za-z])(?=.*\d)[A-Za-z\d@$!%*?&]{8,}$"
return bool(re.match(pattern, password))

# Test cases
passwords = ["mypassword", "mypassword1", "Pass123!", "p@ssw0rd?"]
for pwd in passwords:
    print(f"{pwd}: {validate_password(pwd)}")

```

Explanation

- `(?=.*[A-Za-z])` – Ensures there is at least one letter.
- `(?=.*\d)` – Ensures there is at least one digit.
- `{8,}` – Length is at least 8 characters.

Email Validation

Although email validation can get quite complex (due to RFC standards), a simple pattern often used is:

```

import re

def validate_email(email):
    # A basic pattern for demonstration
    pattern = r"^[a-zA-Z0-9_+-.]+@[a-zA-Z0-9-]+\.[a-zA-Z0-9-.]++$"
    return bool(re.match(pattern, email))

# Test cases
emails = [
    "john.doe@example.com",
    "john_doe123@domain.net",
    "invalid-email@.com",
    "@nodomain.com",
    "just-text"
]

```

```
for e in emails:
    print(f"{e}: {validate_email(e)}")
```

Explanation

- `^[a-zA-Z0-9_+-.]+` – Start with one or more allowed characters before the `@`.
- `@[a-zA-Z0-9-]+` – `@` followed by one or more alphanumeric or hyphen.
- `\.[a-zA-Z0-9-]+\.` – A literal dot (`.`) followed by valid domain characters until the end.

URL Validation

A simplified URL pattern (again, real URLs can be very complex):

```
import re

def validate_url(url):
    # This pattern checks for http or https, optional www., then
    pattern = (
        r"^(https?://)"          # http:// or https://
        r"(www\.)?"             # optional www.
        r"([a-zA-Z0-9-]+)"       # domain name part 1
        r"(\.[a-zA-Z0-9-]+)+"    # domain dot-extension
        r"([/?].*)?$"           # optional path or parameters
    )
    return bool(re.match(pattern, url))

# Test cases
urls = [
    "http://www.google.com",
    "https://example.org/path/to/page?param=value",
    "ftp://not-valid.com",
    "https://domain",
```

```

    "https://sub.domain.co.uk"
]
for u in urls:
    print(f"{u}: {validate_url(u)}")

```

Explanation

- `^(https?://)` – Ensures the URL starts with `http://` or `https://`.
- `(www\.)?` – Optionally allow `www.`.
- `([a-zA-Z0-9-]+)(\.[a-zA-Z0-9-]+)+` – The domain can have multiple subdomains.
- `([/?].*)?$` – Optionally accept a slash or question mark followed by anything until end.

7. Connecting Regex to Lexical Analysis

In compiler design, **lexical analysis** (tokenization) involves converting a stream of characters into a stream of tokens. Each token type (like `IDENTIFIER`, `NUMBER`, `STRING_LITERAL`, `OPERATOR`) can be defined using regular expressions. For example:

- **Identifier:** `^[a-zA-Z_]\w*$` – an identifier must start with a letter or underscore, followed by zero or more word characters.
- **Number (integer):** `^\d+$` – one or more digits.
- **String literal:** `^[^"]*$` – any number of characters (excluding `"`), enclosed in quotes.

Regular expressions help the lexer quickly identify the tokens that appear in the source code before sending them off to syntax analysis, semantic analysis, etc.

8. Conclusion

Regular Expressions are an essential tool for text processing, input validation, and they serve as the foundation for lexical analysis in compilers. By mastering these patterns and Python's `re` module, you can solve a wide range of problems related to parsing and transforming textual data.

Further Reading

- [Python re documentation](#)
- [Regular-Expressions.info](#)
- [Lexical Analysis \(Compiler Design\)](#)

1 Start coding or generate with AI.